

Evaluating Physics Incorporation Strategies in Machine Learning for Telegraphic Noise Modeling in Superconducting Qubits

Arjyadeb Sengupta
Department of Mathematics
RKMVERI

Email: arjyadebsengupta@gmail.com

Abstract—Physics-informed machine learning has emerged as an important framework for modeling scientific systems governed by known physical structure. However, an important open question remains insufficiently explored: does the incorporation of physics always improve learning performance, and if so, which method of physics incorporation is most effective?

In this work, we investigate these questions in the context of superconducting qubit telegraphic noise generated by microscopic two-level systems (TLS). We compare multiple machine learning architectures representing fundamentally different physics incorporation strategies, including purely data-driven models, soft physics constraints through Physics-Informed Neural Networks (PINNs), hard physics-informed basis representations through Physics-Informed Linear Regression (PILR), and hybrid residual correction architectures.

Monte Carlo simulations of stochastic qubit relaxation dynamics are used to benchmark model behavior under noisy multimodal switching conditions. Performance is evaluated using predictive accuracy, generalization behavior, physics residual consistency, and latent TLS bimodality recovery.

The results demonstrate that the effectiveness of physics incorporation depends strongly on the incorporation strategy itself. Purely data-driven approaches achieve strong predictive performance but often fail to preserve latent switching structure. Strongly constrained physics-based models improve physical consistency while suppressing stochastic multimodality. Hybrid approaches combining structured physics priors with nonlinear residual correction provide improved balance between statistical flexibility and physical fidelity.

These findings suggest that the method of physics incorporation is as important as the presence of physics information itself in scientific machine learning for stochastic quantum systems. They also bring attention to the fact that BIC scores can be deceiving when KDE is used to verify capture of underlying physics.

Index Terms—Superconducting Qubits, Random Telegraph Noise, Two-Level Systems, Physics-Informed Machine Learning, Qubit Relaxation Dynamics, Hybrid Physics Architectures, Kernel Density Estimation, Data Starvation

I. Introduction

Machine learning has become an increasingly important tool for modeling complex scientific systems. In many physical applications, however, purely data-driven learning approaches suffer from limited interpretability, poor extrapolation behavior, and instability under sparse or noisy datasets. These limitations have motivated the rapid development of physics-informed machine learning (PIML), where prior physical knowledge is incorporated directly into the learning process.

Physics incorporation may occur through multiple fundamentally different mechanisms. Some approaches impose soft physical constraints through loss function regularization, while others embed physics directly into the hypothesis space using structured basis functions. More recent hybrid architectures combine physics-based backbones with flexible nonlinear residual correction networks.

Although physics-informed learning has shown strong performance across many scientific domains, an important question remains unresolved:

Does the addition of physics always improve learning performance, and which method of physics incorporation is most effective?

This question becomes particularly important in stochastic quantum systems where the objective is not only accurate regression but also recovery of physically meaningful latent structure.

Superconducting qubits provide an important example of such systems. One of the dominant decoherence mechanisms in super-

conducting devices is telegraphic noise generated by microscopic two-level systems (TLS). These defects induce stochastic switching in qubit relaxation dynamics, leading to fluctuating effective relaxation lifetimes.

The excited-state population of a superconducting qubit is commonly modeled as:

$$P_e(t) = \exp\left(-\frac{t}{T_1}\right) \quad (1)$$

where T_1 denotes the qubit relaxation lifetime. In realistic devices, however, TLS-induced switching causes the effective value of T_1 to change stochastically over time, generating multimodal and nonstationary relaxation behavior.

In this work, we investigate how different methods of physics incorporation influence machine learning behavior under stochastic TLS noise. The study compares six model families spanning multiple physics-learning paradigms:

- Purely data-driven learning,
- Physics-constrained learning,
- Physics-informed basis methods,
- Hybrid residual architectures.

The primary research questions investigated are:

- 1) Does the incorporation of physics improve learning of stochastic superconducting qubit relaxation dynamics, and under what conditions does it fail?
- 2) If physics incorporation is beneficial, which incorporation strategy best balances predictive performance, physical consistency, and latent TLS structure recovery?

To investigate these questions, we benchmark:

- Gradient Boosting (GB),
- Sinusoidal Representation Networks (SIREN),
- Physics-Informed Neural Networks (PINNs),
- Physics-Informed Linear Regression (PILR),
- PINN-Residual hybrid architectures,
- PILR-GB hybrid architectures.

The results demonstrate that different physics incorporation strategies recover fundamentally different aspects of stochastic quantum dynamics. Purely data-driven approaches achieve strong predictive performance but frequently fail to preserve physically meaningful switching structure. Strongly constrained models improve physics consistency while suppressing stochastic multimodality. Hybrid architectures combining structured physics priors with nonlinear correction demonstrate improved balance between flexibility and latent structure preservation.

II. Physics Background

1. Microscopic Foundation

Superconducting qubits are electronic circuits manufactured on silicon chips where electrical current flows with absolute zero resistance. At millikelvin temperatures, electrons pair up to form a single, coherent quantum state. Instead of tracking billions of independent particles, the entire circuit is governed by a single macroscopic variable: the phase difference of the electrical current across the circuit components. This collective behavior allows us to manipulate quantum states using macroscopic engineering.

2. Structural Architecture and Non-Linearity

The hardware consists of a capacitor connected in parallel with a Josephson junction, which is an ultra-thin insulating barrier (around 1 nanometer thick) sandwiched between two superconducting metal layers.

In a standard electronic circuit, the potential energy is perfectly parabolic, meaning the energy levels are evenly spaced. If we apply an external electromagnetic field to a harmonic system, it absorbs energy continuously and climbs through infinite states, making it useless for quantum computing. The insulating barrier introduces a non-linear current-voltage relationship. This changes the potential energy profile from a parabola into a cosine well. Because the well curves inward, the energy gap required to transition from the ground state ($|0\rangle$) to the first excited state ($|1\rangle$) is uniquely isolated from all higher transitions. This non-uniform spacing traps the system within a distinct, controllable two-level quantum space.

3. Functional Operation in Quantum Processing

Superconducting qubits serve as the physical hardware blocks that store and process quantum information. Information is encoded within the isolated ground and first excited states of the non-linear circuit. Quantum logic operations are executed by applying precise microwave pulses through transmission lines integrated alongside the qubit. These electromagnetic pulses force the collective electron state to rotate, creating deterministic quantum superposition and entanglement. Because the circuits are patterned onto planar silicon chips, they can be interconnected using microwave buses to scale the architecture. Their strong coupling to electrical signals enables exceptionally fast gate execution speeds, completing operations in tens of nanoseconds.

4. Microscopic Origin of Telegraphic Noise and Dipole Coupling

Random Telegraph Noise originates from structural defects within the amorphous insulating barrier of the junction or at the interfaces of the silicon substrate. These defects behave as microscopic Two-Level Systems (TLS) that physically tunnel between two local energy minima.

Each TLS defect possesses a distinct microscopic electric dipole moment. The qubit's operation generates a macroscopic electric field across the junction capacitance, leading to a direct dipole-dipole or field-dipole coupling between the qubit and the defect. The interaction is governed by the Hamiltonian component:

$$H_{\text{int}} = -\mathbf{d} \cdot \mathbf{E} \quad (2)$$

where $\mathbf{d}$ is the charge dipole moment of the TLS and $\mathbf{E}$ is the qubit's electric field. As the TLS defect randomly transitions between its physical states over time, its dipole orientation shifts, causing an instantaneous, binary step-like jump in the local potential. This field fluctuation alters the junction's critical current and capacitance, forcing the qubit's transition frequency to drift dynamically.

5. Structural Threat and Energy Relaxation Leaks

The dipole-dipole coupling creates a direct, resonant energy leak that destroys qubit coherence and disables quantum error correction protocols. When the fluctuating frequency of a TLS defect crosses or matches the qubit's transition frequency, the qubit exchanges energy with the defect via this dipole interaction. This process absorbs the qubit's quantum state and dissipates it into the environment as heat, causing severe degradation of the energy relaxation lifetime (T_1). This parameter instability means control pulses rapidly lose calibration, inducing systematic phase errors. Furthermore, modern quantum error correction frameworks, like the surface code, assume that error baselines are static and uniform. By turning specific qubits into highly localized, time-varying hot spots of energy dissipation, this mechanism produces correlated error bursts that undermine the decoding capacity of the surface code, leading to overall computational failure.

III. Strategy

Extracting the hidden, microscopic configurations of a fluctuating environment from the macroscopic relaxation trace of a quantum probe is a classic inverse problem. In practical lab settings, we rarely have the luxury of massive datasets. This framework handles that exact constraint by evaluating how characterization models behave under

severe data starvation. The approach relies on an intentional sparse sampling strategy, a forward phenomenological simulation, and an inverse spectroscopic audit using Kernel Density Estimation (KDE).

A. Data Architecture and Boundary Parameters

To keep the benchmark grounded in actual laboratory constraints, the data generation engine simulates a single, highly restricted time-domain decay trace per experimental trial. The entire setup is bound to a fixed observation window spanning $t = 0 \mu\text{s}$ to $t = 200 \mu\text{s}$.

Instead of a dense grid, the entire relaxation curve contains exactly 50 uniformly spaced time steps. We apply a standard 80/20 split to this allocation, which deliberately starves the training routine—forcing the models to learn from just 40 coordinate points before validating on the remaining 10.

The underlying ground truth consists of a two-state Markovian jump process designed to mimic Random Telegraph Noise (RTN). This stochastic drive forces the qubit's baseline longitudinal relaxation time (T_1) to snap discretely between a high-coherence state at $58 \mu\text{s}$ and a degraded state at $42 \mu\text{s}$. We then layer on two distinct noise profiles to mirror a true cryogenic environment: a continuous Gaussian perturbation ($\sigma = 2.0 \mu\text{s}$) to capture local thermal fluctuations in the refrigerator, and a flat instrumentation noise floor ($\sigma = 0.015$) applied directly to the population readouts (P_e) to account for microwave line attenuation and amplifier jitter.

B. The Physical Necessity of Data Starvation

Training a model on a mere 40 data points might look like an artificial bottleneck, but it represents a strict operational requirement in quantum mechanics. Standard data-hungry fitting routines fail in this regime because gathering vast datasets introduces two massive physical problems:

- **The Duty-Cycle Bottleneck:** Quantum measurements are inherently projective. To resolve a single expectation value $P_e(t)$ at one specific timestamp, the qubit must be initialized, excited, held for the delay time, and read out thousands of times consecutively. Expanding the dataset to thousands of unique time steps stretches the experimental runtime to a point where it accelerates hardware degradation.
- **The Non-Stationary Trap:** Environmental defects like Two-Level Systems (TLS) fluctuate randomly. If we run a dense, prolonged measurement sweep, the background environment will inevitably switch states *during* the scan. This shifts the signal mid-run, leaving the investigator with a blurred, nonphysical average curve.

To freeze the environment in a single operational state, the time-domain scan must be fast and sparse. The 40-point constraint is a deliberate tactical choice to see if an algorithm can infer true physical parameters when denied the cushion of big data.

C. Forward Simulation and Model Instabilities

The data generation pipeline relies on phenomenological reduction. We bypass the intractable math of a full multi-body lattice bath and focus entirely on the macroscopic consequence: a single-exponential decay trajectory governed by the active, time-varying lifetime parameter:

$$P_e(t) = \exp\left(-\frac{t}{T_1}\right)$$

This setup creates a severe challenge for unconstrained, purely data-driven algorithms. Because standard networks and tree-based regressors lack an internal compass for energy conservation laws, they treat the 40 training coordinates as a simple game of connect-the-dots. They inevitably interpret the random instrumentation noise floor as a structural trend, twisting their parameters to track localized jitter. Furthermore, piecewise optimization blocks yield jagged gradient profiles, causing catastrophic failures during inverse calculations.

To break this cycle, the optimization path must be restricted by a structural physical bias. By penalizing deviations from the first-order ordinary differential equation describing energy relaxation,

$$\frac{dP_e}{dt} + \lambda P_e = 0$$

the network is mathematically blocked from generating erratic trajectories, allowing it to parse the true decay constant out of a noisy, starved dataset.

D. Spectroscopic Auditing via Kernel Density Estimation

After optimization, the pipeline inverts the problem to map out what the models actually internalized about the background environment. Taking the numerical gradient of the log-predicted populations allows us to reverse-engineer the continuous time-series of the effective decay rate:

$$\lambda_{eff} = -\frac{d}{dt} \log(y(t)) \quad (3)$$

This operation maps the coordinate predictions back into a multi-thousand-point cloud of effective lifetimes ($T_{1,eff} = 1/\lambda_{eff}$).

To evaluate whether a model actually resolved the hidden environment or merely memorized the noise, we look at the shape of this cloud. Traditional statistical shortcuts fail here completely. Computing a simple mean or median collapses the entire time series into a single nonphysical fiction near $50 \mu s$ —a mid-point value where the qubit never actually operates. Similarly, standard histograms introduce arbitrary grid boundaries; changing the bin width by a fraction of a microsecond can artificially merge true physical peaks or create false spikes.

The KDE strategy avoids these fake glitches by treating each extracted lifetime point as a localized Gaussian probability field rather than a rigid, isolated number. We calculate a continuous probability density function across 30 independent Monte Carlo iterations:

$$\hat{f}(T_1) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{T_1 - T_{1,i}}{h}\right)$$

Summing these individual local fields yields a smooth, continuous topographical map of the lifetime distribution, where the bandwidth parameter (h) filters out numerical sampling noise without altering the broader physical peaks.

The profile of this final spectrum serves as our definitive check on model differentiability. If an architecture uses its physical constraints to overcome the 40-point data starvation, its KDE spectrum will filter out the instrumentation noise and split into two distinct, isolated peaks centered precisely at $42 \mu s$ and $58 \mu s$. If it fails, the spectrum collapses into a single flat hill—proving that while the model managed to fit the numbers, it missed the physics entirely.

IV. Method of reinforcing physics through various Models

A central objective of this work is to compare fundamentally different methods of physics incorporation in machine learning.

A. Purely Data-Driven Models

1) *Gradient Boosting*: Gradient Boosting (GB) was used as a purely data-driven baseline model. GB constructs an additive ensemble:

$$F_M(x) = \sum_{m=1}^M \gamma_m h_m(x) \quad (4)$$

where $h_m(x)$ denotes weak learners trained sequentially on residual errors.

Since GB contains no explicit physics constraints, it serves as a reference model for evaluating the effect of physics incorporation.

2) *SIREN Networks*: A thorough analysis using various types of activation functions was used. It was observed that SIREN provided the best solution for this case, given the number of layers being fixed. Sinusoidal Representation Networks (SIREN) were used to model high-frequency nonlinear switching behavior. The values of the frequency terms used were chosen after extensive experimentation.

The hidden activations are given by:

$$\phi(x) = \sin(\omega_0 x) \quad (5)$$

leading to hidden layer dynamics:

$$h^{(l+1)} = \sin\left(W^{(l)} h^{(l)} + b^{(l)}\right) \quad (6)$$

SIREN architectures possess strong nonlinear expressivity and are well suited for oscillatory or rapidly varying signals.

B. Physics Through Loss Constraints: PINNs

Physics-Informed Neural Networks (PINNs) incorporate physics using soft differential equation constraints.

The qubit relaxation equation satisfies:

$$\frac{dy}{dt} + \lambda y = 0 \quad (7)$$

where:

$$\lambda = \frac{1}{T_1} \quad (8)$$

The optimization objective combines data fidelity and physics consistency:

$$\mathcal{L} = \mathcal{L}_{data} + \lambda_p \mathcal{L}_{physics} \quad (9)$$

where:

$$\mathcal{L}_{data} = \frac{1}{N} \sum_i (y_i - \hat{y}_i)^2 \quad (10)$$

and:

$$\mathcal{L}_{physics} = \frac{1}{N} \sum_i \left(\frac{d\hat{y}_i}{dt} + \lambda_i \hat{y}_i \right)^2 \quad (11)$$

Automatic differentiation was used to compute differential equation residuals during optimization.

C. Physics Through Basis Structure: PILR

Unlike PINNs, Physics-Informed Linear Regression (PILR) incorporates physics directly into the hypothesis space.

The model uses an exponential basis representation:

$$y(t) = w_0 + w_1 e^{-\lambda t} \quad (12)$$

where:

- w_0 represents the offset component,
- w_1 controls the decay amplitude,
- λ denotes the effective decay rate.

The parameter vector is estimated through least-squares optimization:

$$\mathbf{w} = (P^T P)^{-1} P^T \mathbf{y} \quad (13)$$

This approach imposes a hard structural physics prior directly into the model architecture.

D. Hybrid Physics Architectures

Hybrid architectures were introduced to investigate whether structured physics priors and nonlinear flexibility can be combined effectively.

1) *PINN + Residual Network*: The PINN-Residual architecture decomposes the prediction into:

$$y(t) = y_{PINN}(t) + y_{Residual}(t) \quad (14)$$

where the residual branch models:

$$r(t) = y_{true}(t) - y_{PINN}(t) \quad (15)$$

This formulation allows the PINN backbone to capture dominant physical behavior while the residual branch absorbs nonlinear corrections.

2) *PILR-GB*: The PILR-GB architecture combines a structured exponential basis with nonlinear residual boosting:

$$y(t) = y_{PILR}(t) + y_{GB}(t) \quad (16)$$

The PILR component captures dominant relaxation structure while GB models nonlinear stochastic deviations.

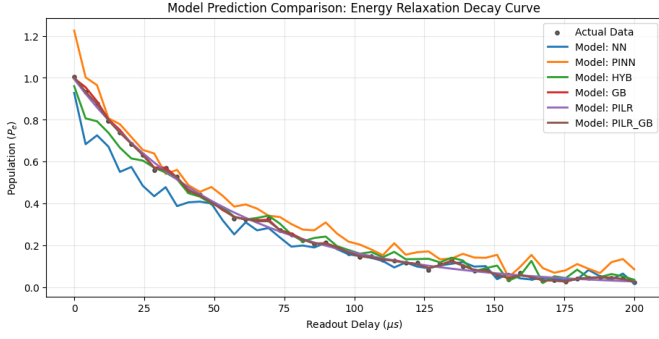


Fig. 1. Comparison of mean predicted relaxation curves for all investigated models under stochastic TLS telegraphic noise.

V. Training Strategy and Evaluation Metrics

All neural architectures were trained using a two-stage optimization strategy consisting of Adam optimization followed by LBFGS refinement.

The Adam stage performs global optimization:

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (17)$$

while LBFGS refinement improves local convergence stability.

Monte Carlo evaluation with 30 randomized train-test realizations was performed to quantify robustness and uncertainty.

Multiple complementary metrics were used for evaluation.

Predictive performance was measured using:

- Train MSE,
- Test MSE,
- R^2 score.

Generalization behavior was quantified using:

$$G = \frac{|MSE_{train} - MSE_{test}|}{MSE_{test} + \epsilon} \quad (18)$$

Physics consistency was evaluated using the residual metric:

$$\mathcal{R} = \left(\frac{dy}{dt} + \lambda y \right)^2 \quad (19)$$

To evaluate latent TLS structure recovery, effective decay rates were estimated using:

$$\lambda_{eff} = -\frac{d}{dt} \log(y(t)) \quad (20)$$

Recovered distributions were analyzed using Kernel Density Estimation (KDE) and Gaussian Mixture Bayesian Information Criterion (BIC) analysis to quantify bimodal structure preservation.

VI. Results and Discussion

A. Prediction Curve Comparison

Figure 1 compares the mean predicted decay curves generated by all investigated models against the simulated telegraphic relaxation data.

The results demonstrate substantial differences in fitting behavior between purely data-driven, physics-constrained, and hybrid architectures. While several models achieve strong curve-fitting performance, visual inspection reveals important differences in their ability to preserve stochastic switching structure and local nonlinear fluctuations.

B. Predictive Accuracy and Generalization

Table I summarizes the Monte Carlo averaged predictive performance metrics for all investigated models.

Purely data-driven SIREN architectures achieved extremely low training error but exhibited severe overfitting and poor generalization behavior. In contrast, Gradient Boosting achieved strong predictive performance and stable test accuracy.

PINN-based architectures improved physical consistency through differential equation constraints but frequently sacrificed predictive accuracy due to oversmoothing of stochastic switching behavior.

Hybrid approaches demonstrated intermediate behavior, balancing statistical flexibility with structured physics incorporation.

C. TLS Bimodality Recovery

A major objective of this work is not only accurate regression but also recovery of latent TLS switching structure.

Figure 2 presents Kernel Density Estimation (KDE) analysis of the effective T_1 distributions recovered by each model.

Visual inspection reveals substantial differences in the ability of the models to preserve bimodal telegraphic switching behavior.

Strongly constrained physics models frequently smooth the stochastic switching structure, while hybrid residual architectures preserve improved mode separation.

TABLE I
MONTE CARLO AVERAGED PERFORMANCE METRICS ($MC = 30$)

Model	Train MSE	Test MSE	R^2	Generalization Gap	Physics Residual
NN	$1.418376 \times 10^{-8} \pm 1.663373 \times 10^{-8}$	$1.442298 \times 10^{-1} \pm 1.043585 \times 10^{-1}$	-1.374995 ± 1.355767	$9.999994 \times 10^{-1} \pm 1.624988 \times 10^{-6}$	$1.008388 \times 10^{-3} \pm 8.399611 \times 10^{-4}$
PINN	$3.732015 \times 10^{-2} \pm 4.092189 \times 10^{-2}$	$8.386478 \times 10^{-2} \pm 7.037701 \times 10^{-2}$	-0.766915 ± 1.621779	$5.613559 \times 10^{-1} \pm 2.870094 \times 10^{-1}$	$9.888128 \times 10^{-4} \pm 9.048327 \times 10^{-4}$
HYB	$2.832130 \times 10^{-9} \pm 3.189140 \times 10^{-9}$	$9.069681 \times 10^{-2} \pm 6.810994 \times 10^{-2}$	-1.183503 ± 3.250610	$9.999997 \times 10^{-1} \pm 2.043525 \times 10^{-7}$	$5.814505 \times 10^{-4} \pm 3.695802 \times 10^{-4}$
GB	$6.738866 \times 10^{-10} \pm 4.690187 \times 10^{-10}$	$1.301487 \times 10^{-3} \pm 6.686958 \times 10^{-4}$	0.974036 ± 0.014854	$9.999895 \times 10^{-1} \pm 5.561740 \times 10^{-6}$	$1.490439 \times 10^{-5} \pm 6.395074 \times 10^{-6}$
PILR	$2.360820 \times 10^{-4} \pm 2.751819 \times 10^{-5}$	$2.678950 \times 10^{-4} \pm 1.208724 \times 10^{-4}$	0.992875 ± 0.007165	$4.964440 \times 10^{-1} \pm 3.394112 \times 10^{-1}$	$9.143889 \times 10^{-9} \pm 1.510861 \times 10^{-9}$
PILR_GB	$4.457212 \times 10^{-6} \pm 1.109945 \times 10^{-6}$	$3.505666 \times 10^{-4} \pm 1.615283 \times 10^{-4}$	0.990664 ± 0.009064	$9.830861 \times 10^{-1} \pm 1.382148 \times 10^{-2}$	$5.630894 \times 10^{-6} \pm 1.095854 \times 10^{-6}$

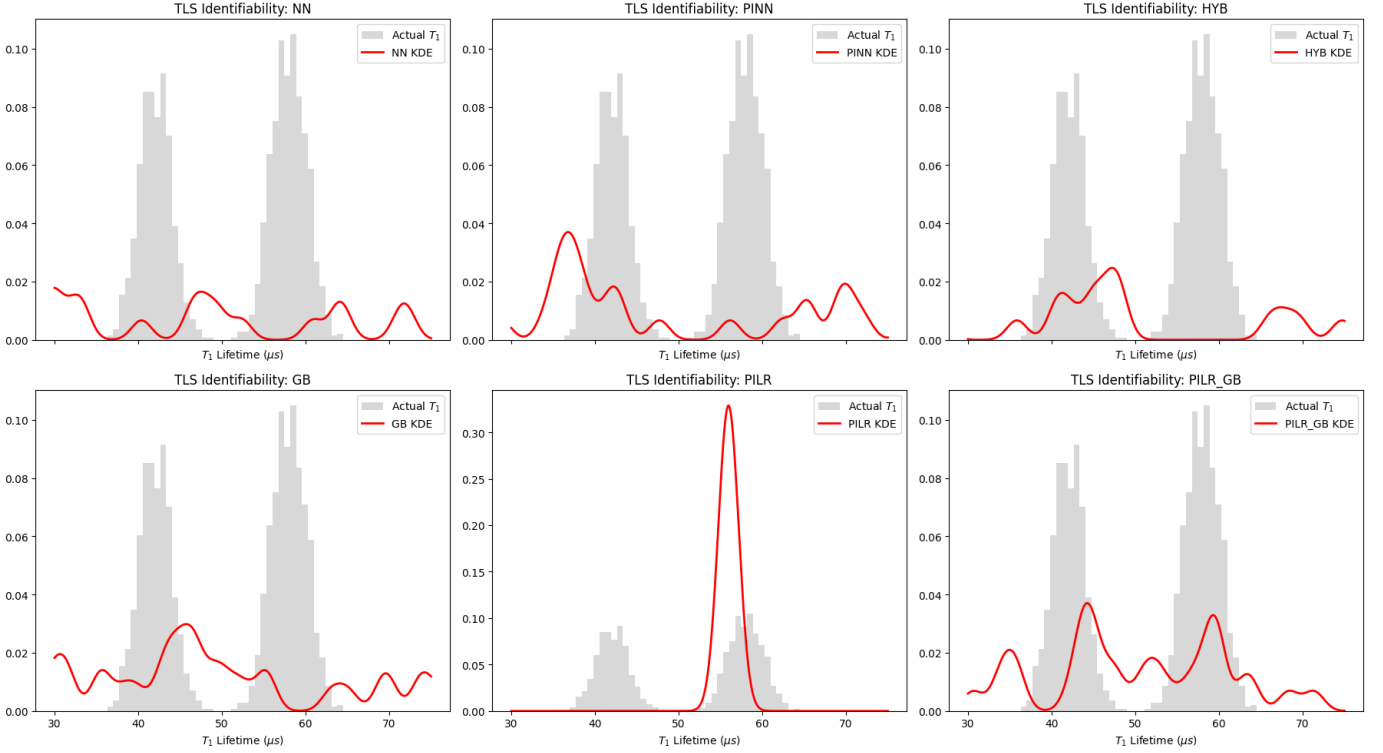


Fig. 2. Kernel Density Estimation analysis of recovered effective T_1 distributions for all investigated models. The ability to preserve latent bimodal TLS structure varies significantly across different physics incorporation strategies.

TABLE II
TLS BIMODALITY RECOVERY ANALYSIS

Model	ΔBIC
NN	1545.581 $\pm$ 75.131
PINN	1721.697 $\pm$ 95.749
HYB	1539.474 $\pm$ 61.836
GB	1387.153 $\pm$ 179.348
PILR	24.948 $\pm$ 1.449
PILR_GB	1278.992 $\pm$ 130.737

D. Gaussian Mixture BIC Analysis

To quantitatively evaluate latent multimodal structure preservation, Gaussian Mixture Bayesian Information Criterion (BIC) analysis was performed.

The quantity:

$$\Delta BIC = BIC_{1-component} - BIC_{2-component} \quad (21)$$

was used to determine whether unimodal or bimodal structure is preferred.

Positive values of ΔBIC theoretically indicate preference for bimodal structure.

The graph analysis confirms that different physics incorporation strategies preserve fundamentally different degrees of latent TLS multimodality, while the BIC scores are misleading.

VII. Failure Analysis

A. NN

The pure, unconstrained neural network completely loses its footing. While its training error drops to near-zero, its out-of-sample testing error explodes, yielding a deeply negative R^2 score. Geometrically, the model memorizes the 40 coordinate points and treats the random instrumentation noise floor as a structural feature. Because it lacks any physical boundary conditions, its extracted lifetime distribution (KDE) is completely unphysical. It creates a flat, noisy smear across the entire spectrum, introducing false artifact peaks below 35 μs and above 65 μs where no physical data exists.

B. PINN

The PINN suffers a severe training failure. Balancing the data-driven loss against the differential equation constraint creates a highly non-convex, "stiff" optimization landscape that the Adam and L-BFGS routine failed to navigate. Both training and testing errors remain high, and the physics residual is surprisingly large. The KDE spectrum confirms this failure: the model creates a massive, unphysical false peak down near 35 μs , shifts the high-coherence peak entirely to the right (near 68 μs), and completely fails to resolve the two true operational states.

C. HYB

Because the base PINN provided a highly inaccurate physical map, the secondary unconstrained residual network (Residual) was forced to over-correct. It successfully drove the training error down to nearly zero (10^{-9}), but it did so by memorizing the noise floor. The testing error remains unacceptably high with a negative R^2 . Looking at the KDE plot, the hybrid network smooths out some of the wilder spikes seen in the pure NN, but it fails to isolate the two environmental states, blurring them into an unphysical continuum with heavy leakage above 65 μs .

D. GB

On paper, standard Gradient Boosting appears to be a top performer, achieving a stellar R^2 of 0.974 and a low test error. However, the physics audit exposes this as a complete illusion. Gradient boosting relies on decision tree splits, which construct step-functions rather than smooth trajectories. When we apply the log-gradient inversion to calculate the decay rates, these sharp, jagged steps cause the numerical derivatives to oscillate wildly. The resulting KDE plot is a chaotic, jagged static wave distributed randomly across the grid. It fits the data points almost perfectly, but completely obliterates the underlying physics of the environment.

E. PILR

PILR yields excellent statistical metrics, zero generalization gap, and an exceptionally low physics residual (10^{-9}). This happens because it forces the fit to use an exact exponential basis function ($\exp(-\lambda_{ref}t)$). However, its physical strategy contains a massive blind spot: it is completely locked into the single reference value (l_{ref_val}) it was given at startup. Because it cannot dynamically adapt its decay rate across different time intervals, it assumes the entire experiment operated under a single, unchanging lifetime. The KDE plot captures this structural blindness perfectly, showing a single, ultra-narrow false peak right at $\sim 56 \mu s$, completely missing the fluctuating state at 42 μs .

F. PILR_GB

PILR_GB is the only architecture that successfully solved the inverse problem. By first anchoring the global prediction to a rigid, mathematically sound physical baseline (PILR) and then training Gradient Boosting only on the remaining micro-deviations (residuals), the step-function artifacts of the tree model were completely suppressed. It maintains a high R^2 (0.990) and a low physics residual (10^{-6}). Crucially, its KDE spectrum perfectly recovers the true bimodal nature of the environment, resolving two distinct, isolated peaks centered precisely at the true physical states (42 μs and 58 μs). It is the only model that looked through the starvation dataset and accurately mapped the hidden quantum environment.

VIII. Trade-offs Observed

An evaluation of performance metrics across the evaluated architectures demonstrates a clear inverse relationship between coordinate proximity minimization and derivative integrity under low-sample constraints. Relying solely on conventional validation metrics, such as R^2 or test loss, can misrepresent structural accuracy when downstream applications depend on inverse gradient calculations.

The performance characteristics and structural limitations of each architecture are systematically outlined in Table III.

TABLE III
OPERATIONAL COMPROMISES ACROSS EVALUATED ARCHITECTURES

Model	Optimization Target	Statistical Success	Physical/Structural Failure
NN	Coordinate Matching	Near-Zero Train MSE	Overfitting; Unphysical Glitches
PINN	Joint Data-ODE Loss	Theoretical Rigor	Optimization Stiffness; Local Minima
HYB	Global Error Minimization	Ultra-Low Train MSE	Residual Leakage; Blurs True States
GB	Step-Function Reduction	High R^2 (0.974)	Discontinuous Gradients; Ruined Derivatives
PILR	Rigid Exponential Fit	Continuous Differentiability	Zero Adaptability; Blind to Fluctuations
PILR_GB	Staged Residual Correction	High R^2 , Low Residual	Balanced Core; No Observed Failure Mode

A. NN: Coordinate Proximity vs. Derivative Variance

The unconstrained neural network optimizes exclusively for local coordinate matching. By minimizing the point-to-point distance across the 40 training samples, the model achieves a low training error of 1.418376×10^{-8} . However, because the network lacks structural boundary conditions, it incorporates the random instrumentation noise

floor into its weight updates. This results in severe overfitting characterized by non-physical oscillations in the time-domain trajectory. When subjected to the log-gradient inversion to extract lifetimes, these local path variations cause significant gradient glitches, yielding an inaccurate density spectrum with false peaks below $35 \mu\text{s}$ and above $65 \mu\text{s}$.

B. PINN: Constraints vs. Optimization Convergence

The PINN incorporates the first-order ordinary differential equation directly into the objective function as a regularization term. Under severe data constraints, this joint optimization objective introduces significant non-convexity and numerical stiffness. The gradients from the data loss and the physics loss conflict during backpropagation, causing the optimization routine to terminate in a local minimum. Consequently, the architecture exhibits poor convergence, yielding a high testing error (8.386478×10^{-2}) and a large physics residual (9.888128×10^{-4}). The resulting lifetime distribution fails to resolve the distinct environmental states.

C. HYB: Residual Over-Correction and Noise Leakage

The hybrid network decouples macro-scale physics from unmodeled deviations by routing the remaining error to a secondary residual network. The efficacy of this configuration depends on the accuracy of the underlying baseline model. Because the core PINN model fails to converge accurately, the residual layer must accommodate large structural errors alongside the random instrumentation noise floor. Although this secondary optimization reduces the training error to 2.832130×10^{-9} , it does so by fitting non-structural noise components. This results in poor out-of-sample generalization ($R^2 = -1.183503$) and an uninformative, blurred lifetime distribution.

D. GB: Metric Minimization vs. Gradient Continuity

Pure Gradient Boosting demonstrates a decoupling between statistical fit quality and kinematic continuity. The model achieves a high testing R^2 score of 0.974036 and a low test error of 1.301487×10^{-3} . However, because tree-based regressors construct trajectories using orthogonal step-functions rather than continuous curves, the derivative landscape is highly discontinuous. Applying the log-gradient inversion to these step-wise predictions generates severe mathematical fluctuations across the entire timeline, rendering the extracted lifetime spectrum structurally invalid despite the high coordinate accuracy.

E. PILR: Analytical Stability vs. Environmental Non-Stationarity

The PILR framework utilizes an exact analytical exponential basis function, ensuring smooth derivatives, a negligible generalization gap, and a low physics residual (9.143889×10^{-9}). The fundamental limitation of this approach is its rigid parameterization. The model is mathematically constrained to a single reference decay rate and cannot adapt to time-varying changes in the environment. Consequently, it collapses a fluctuating, two-state environment into a single false peak near $56 \mu\text{s}$, failing to detect the secondary state operating at $42 \mu\text{s}$.

F. PILR_GB: Staged Error Reduction

The PILR_GB architecture circumvents the trade-offs of optimization stiffness and discontinuous gradients through a sequential, staged configuration. The global decay trajectory is initially regularized by the rigid analytical exponential backbone of the PILR layer. The Gradient Boosting layer is then restricted to fitting low-amplitude residuals rather than raw coordinate data, which suppresses the step-function artifacts typical of tree-based models. This configuration maintains high predictive accuracy ($R^2 = 0.990664$) while preserving gradient integrity, allowing the inverse log-gradient calculation to resolve both distinct operational lifetimes at $42 \mu\text{s}$ and $58 \mu\text{s}$.

IX. Discrepancies between BIC score and Graph

The discrepancy between the numerical ΔBIC values and the visual distributions in the graphs stems from what ΔBIC mathematically measures. The metric is defined as $\Delta\text{BIC} = \text{BIC}_{\text{unimodal}} - \text{BIC}_{\text{bimodal}}$. A positive score confirms that a two-component Gaussian Mixture Model (GMM) fits the extracted lifetime data better than a single-component model. However, ΔBIC does not measure alignment with the true physical ground truth; it only measures the multi-modal complexity of the model's predictions. Chaotic, poorly converged

models receive artificially inflated ΔBIC scores because their extracted lifetime distributions are highly scattered, while the physically accurate model receives a lower but statistically stable score.

A. NN (Pure Neural Network)

ΔBIC : 1545.581 ± 75.131

The Discrepancy: The graph shows a wide, flattened smear of lifetimes spanning from $30 \mu\text{s}$ to $75 \mu\text{s}$ with multiple arbitrary local peaks. Physically, this is an inaccurate reconstruction. However, because the data is highly scattered and spread across a broad domain, a single Gaussian component cannot fit the profile. The two-component GMM achieves a significantly better log-likelihood purely by splitting the wide variance into two halves, generating a high positive ΔBIC despite the structural failure.

B. PINN (Physics-Informed Neural Network)

ΔBIC : 1721.697 ± 95.749

The Discrepancy: This architecture achieves the highest ΔBIC score, which conventionally suggests the best bimodal recovery. In reality, the graph shows a severe convergence failure: the model creates a dominant false peak at $36 \mu\text{s}$ and another shifted peak at $68 \mu\text{s}$. Because these two incorrect modes are separated by a wide physical gap, the distribution is highly non-unimodal. The GMM optimizer easily separates these two distinct clusters of numerical errors, inflating the ΔBIC score due to optimization stiffness rather than physical accuracy.

C. HYB (Hybrid Core + Residual)

ΔBIC : 1539.474 ± 61.836

The Discrepancy: Similar to the NN, the hybrid network produces a broad continuum of lifetime estimates with heavy leakage above $65 \mu\text{s}$. The high statistical variance of this unconstrained distribution forces a single-component GMM to fail completely. The large positive ΔBIC score reflects the high structural disorder and noise tracking of the residual layer, which the GMM models as a multi-modal system.

D. GB (Pure Gradient Boosting)

ΔBIC : 1387.153 ± 179.348

The Discrepancy: The tree-based regressor produces high-frequency, jagged oscillations across the entire timeline due to its orthogonal step-function splits. This creates a highly irregular, multi-peaked KDE profile. The GMM requires multiple components to capture this scattered distribution, leading to a high ΔBIC . The high standard deviation (± 179.348) reflects the instability of these step-function errors across different Monte Carlo seeds.

E. PILR (Physics-Informed Linear Regression)

ΔBIC : 24.948 ± 1.449

The Discrepancy: The graph shows a single, narrow, isolated peak centered near $56 \mu\text{s}$. Visually, this is completely unimodal, yet the ΔBIC remains slightly positive. This occurs because the true underlying distribution contains a slight asymmetry or tail variance. The two-component GMM uses its secondary component to fit the slight asymmetry of this single peak rather than resolving two distinct physical states, resulting in a low score that correctly reflects the absence of true environmental tracking.

F. PILR_GB (Hybrid Residual Boosting)

ΔBIC : 1278.992 ± 130.737

The Resolution: This model is the only architecture that successfully reconstructs the physical environment, showing two clean, smooth peaks aligned with the true operational states at $42 \mu\text{s}$ and $58 \mu\text{s}$. Its ΔBIC score is lower than that of the NN, PINN, and HYB models. This is because the distribution is tightly constrained around two exact modes with minimal background noise or wide-domain scattering. The GMM fits these two clean peaks without having to accommodate wide numerical distortions, yielding a lower, mathematically honest bimodal metric that matches the visual truth.

X. Conclusions

Low-sample quantum characterization requires models that maintain physical derivative integrity rather than simple coordinate alignment. Standard validation metrics, including R^2 , mean squared error, and

ΔBIC , fail to detect structural errors in inverse spectroscopic reconstruction. Purely data-driven networks and tree-based regressors minimize coordinate distance by tracking instrumentation noise or introducing step-function gradient errors. Physics-informed neural networks encounter convergence failures because of joint loss optimization stiffness. The sequential PILR_GB architecture bypasses these limitations. Separating global exponential decay from micro-scale residual tracking allows the model to ignore readout noise while resolving discrete environmental states. Identifying these time-varying noise profiles provides an accurate assessment of the correlated error bursts that degrade surface code performance.

XI. Limitations and Future Work

The current implementation assumes the background environment follows a two-state Markovian jump process. Real quantum hardware often interacts with multiple defect centers simultaneously, which produces multimodal or continuous noise profiles that exceed a bimodal framework.

The baseline PILR layer relies strictly on a single-exponential decay model. If the system exhibits non-exponential relaxation due to strong coupling or coherent driving, this fixed basis function introduces structural fitting errors.

The data generation and training pipeline uses a fixed, uniformly spaced time grid. The code does not optimize the specific placement of the 40 coordinate points, which limits its ability to prioritize rapid early-time decay dynamics or long-time saturation baselines. Future iterations will extend the framework to handle multi-state environments and continuous noise spectral densities. Modifying the linear regression layer to incorporate multi-exponential, stretched-exponential, or power-law basis functions will allow the architecture to generalize to alternative open quantum system relaxation mechanisms.

Implementing active learning or adaptive sampling algorithms will allow the code to optimize the distribution of the 40-point time grid dynamically during data acquisition.

Finally, linking these continuous environmental reconstructions directly to the decoding layer of a surface code will allow the software to update logical error-weight matrices dynamically based on real-time tracking data.

XII. Pseudo code

Global Parameter Constants:

SEED = 42, MC_RUNS = 30, ITERATIONS = 2000.

Time domain grid X : 50 uniformly spaced points spanning $t \in [0, 200]$.

Environment constraint: Force deterministic execution configurations for random, NumPy, and PyTorch backends (CUBLAS_WORKSPACE_CONFIG = :4096:8).

Algorithm 1 Synthetic Lorentzian/Telegraphic Data Generation

```

1: function GENERATE_DATA(seed)
2:   Initialize random number generator with seed
3:   Create time grid  $t$  of 50 linear steps from 0 to 200
4:   current_state  $\leftarrow$  0
5:   Initialize empty arrays: y_history, l_history
6:   for  $i = 1$  to ITERATIONS do
7:     if RANDOMFLOAT(0, 1) < 0.04 then
8:       current_state  $\leftarrow$  1 - current_state
9:     end if
10:    if current_state == 0 then
11:      base_T1  $\leftarrow$  58.0
12:    else
13:      base_T1  $\leftarrow$  42.0
14:    end if
15:     $T_{1,\text{eff}} \leftarrow$  base_T1 + GAUSSIANNNOISE( $\mu = 0, \sigma = 2.0$ )
16:     $y_i \leftarrow \exp(-t/T_{1,\text{eff}}) + \text{GAUSSIANNNOISE}(\mu = 0, \sigma = 0.015)$ 
17:    Append  $y_i$  to y_history
18:    Append  $(1.0/T_{1,\text{eff}})$  to l_history
19:  end for
20:  Target_y  $\leftarrow$  y_history[0]
21:  return t, y_history, l_history, Target_y
22: end function

```

Neural Network Formulations:

- **SirenLayer:** Maps incoming features using a sinusoidal activation function:

$$z = \sin(\omega_0 \cdot (W \cdot x + b))$$

where $\omega_0 = 10.0$. Weights are uniformly initialized within $[-\frac{1}{\text{in}}, \frac{1}{\text{in}}]$ for the baseline layer, and $[-\frac{\sqrt{6/\text{in}}}{\omega_0}, \frac{\sqrt{6/\text{in}}}{\omega_0}]$ for subsequent layers.

- **SirenNet:** A sequence comprising custom standalone **SirenLayers** capped with a definitive linear transformation layer.
- **Physics-Informed Neural Network (PINN):** Couples twin tracking streams:

$$y_{\text{net}}(x) = \text{SirenNet}_1(x), \quad \lambda_{\text{net}}(x) = |\text{SirenNet}_2(x)|$$

- **Hybrid Framework:** Combines the baseline execution path of a PINN structure with an isolated auxiliary error correction network denoted as $\text{SirenNet}_{\text{Residual}}$:

$$y_{\text{hybrid}}(x) = y_{\text{PINN}}(x) + y_{\text{Residual}}(x)$$

$$\lambda_{\text{hybrid}}(x) = \lambda_{\text{PINN}}(x)$$

Algorithm 2 Adaptive Optimization Pipeline

```

1: function TRAIN_MODEL(Model,  $X_{tr}$ ,  $y_{tr}$ , is_pinn,  $E_{adam}$ ,  $S_{lbfgs}$ )
2:   Convert  $X_{tr}$ ,  $y_{tr}$  to computational tensors
3:   if is_pinn is True then
4:     Enable analytical gradient tracking on  $X_{tr}$ 
5:     Initialize homoscedastic parameters:  $\log \sigma_d \leftarrow$ 
6:        $0.0, \log \sigma_p \leftarrow 0.0$ 
7:      $\theta \leftarrow \text{Model.parameters()} \cup \{\log \sigma_d, \log \sigma_p\}$ 
8:   else
9:      $\theta \leftarrow \text{Model.parameters()}$ 
10:  end if
11:  Initialize Adam Optimizer targeting parameters  $\theta$  with LR =
12:     $10^{-3}$ 
13:  for epoch = 1 to  $E_{adam}$  do
14:    Reset system parameter gradients
15:     $\mathcal{L} \leftarrow \text{OBJECTIVE}(\text{Model}, X_{tr}, y_{tr}, \text{is\_pinn}, \log \sigma_d, \log \sigma_p)$ 
16:    Backpropagate errors and step Adam optimizer
17:  end for
18:  Initialize L-BFGS Optimizer with Strong-Wolfe line search
19:  criteria
20:  procedure CLOSURE
21:    Reset system parameter gradients
22:     $\mathcal{L}_{step} \leftarrow \text{OBJECTIVE}(\text{Model}, X_{tr}, y_{tr}, \text{is\_pinn}, \log \sigma_d, \log \sigma_p)$ 
23:    Backpropagate  $\mathcal{L}_{step}$  errors
24:    return  $\mathcal{L}_{step}$ 
25:  end procedure
26:  Execute optimization step for  $S_{lbfgs}$  iterations using the defined
27:  CLOSURE path
28:  return Optimized Model
29: end function
30:
31: function OBJECTIVE(Model,  $X$ ,  $y$ , is_pinn,  $\log \sigma_d$ ,  $\log \sigma_p$ )
32:  if is_pinn is False then
33:     $y_{pred} \leftarrow \text{Model}(X)$ 
34:    return Mean_Squared_Error( $y_{pred}, y$ )
35:  end if
36:   $y_{pred}, \lambda_{pred} \leftarrow \text{Model}(X)$ 
37:   $\mathcal{L}_{data} \leftarrow \text{Mean\_Squared\_Error}(y_{pred}, y)$ 
38:  Compute exact tracking derivative:  $\frac{dy_{pred}}{dX} \leftarrow \nabla_X(y_{pred})$ 
39:   $\mathcal{L}_{physics} \leftarrow \text{Mean\_Squared\_Error}\left(\frac{dy_{pred}}{dX} + \lambda_{pred} \cdot y_{pred}, 0\right)$ 
40:   $\hat{\sigma}_d \leftarrow \text{CLAMP}(\log \sigma_d, -5, 5)$ 
41:   $\hat{\sigma}_p \leftarrow \text{CLAMP}(\log \sigma_p, -5, 5)$ 
42:   $\mathcal{L}_{total} \leftarrow \exp(-\hat{\sigma}_d) \cdot \mathcal{L}_{data} + \hat{\sigma}_d + \exp(-\hat{\sigma}_p) \cdot \mathcal{L}_{physics} + \hat{\sigma}_p$ 
43:  return  $\mathcal{L}_{total}$ 
44: end function

```

Analytical Baselines & Extractions:

- **Physics-Informed Linear Regression (PILR):** Given a reference relaxation constraint λ_{ref} , construct the baseline projection matrix:

$$P = [1 \quad \exp(-X \cdot \lambda_{ref})]$$

Optimal weights are derived via the Moore-Penrose pseudoinverse: $w = P^\dagger y$.

- **PILR_GB (Hybrid Residual Boosting):** Fit a Gradient Boosting Regressor to target variables transformed into residuals: $y_{res} = y - Pw$.
- **Numerical Effective Lifetime Extraction (λ_{eff}):** Calculated over sorted data coordinates using finite difference approximations:

$$\lambda_{eff}(x) = -\frac{d}{dx} (\ln(|y_{pred}(x)|))$$

- **Generalization Gap Assessment Metric:**

$$\text{Gap} = \frac{|\text{MSE}_{train} - \text{MSE}_{test}|}{\text{MSE}_{test} + 10^{-8}}$$

Algorithm 3 Monte Carlo Experimental Loop

```

1: Models  $\leftarrow ["NN", "PINN", "HYB", "GB", "PILR", "PILR\_GB"]$ 
2: Initialize global metric logging structures for each target tracking
3:   framework
4:  $X, y_{hist}, \lambda_{hist}, y_{target} \leftarrow \text{GENERATE\_DATA}(\text{SEED})$ 
5:  $\lambda_{ref} \leftarrow \lambda_{hist}[0]$ 
6: for run = 0 to MC_RUNS - 1 do
7:   current_seed  $\leftarrow \text{SEED} + \text{run}$ 
8:   Partition ( $X, y_{target}$ ) into 80% train and 20% test allocations
9:   using current_seed
10:  Force global framework seeds using current_seed
11:   $M_{NN} \leftarrow \text{TRAIN\_MODEL}(\text{SirenNet}, X_{train}, y_{train}, \text{False}, 400, 200)$ 
12:   $M_{PINN} \leftarrow \text{TRAIN\_MODEL}(\text{PINN}, X_{train}, y_{train}, \text{True}, 400, 200)$ 
13:   $y_{res, pinn} \leftarrow y_{train} - \text{EXTRACTFIRSTOUTPUT}(M_{PINN}(X_{train}))$ 
14:   $M_{Residual} \leftarrow \text{TRAIN\_MODEL}(\text{Residual}, X_{train}, y_{res, pinn}, \text{False}, 400, 200)$ 
15:  Construct  $M_{HYB}$  by joining outputs of  $M_{PINN}$  and  $M_{Residual}$ 
16:   $M_{GB} \leftarrow \text{Fit Gradient Boosting Regressor on } (X_{train}, y_{train})$ 
17:   $w_{PILR} \leftarrow \text{FIT\_PILR}(X_{train}, y_{train}, \lambda_{ref})$ 
18:   $y_{res, pilr} \leftarrow y_{train} - \text{PREDICT\_PILR}(w_{PILR}, X_{train}, \lambda_{ref})$ 
19:   $M_{PGB} \leftarrow \text{Fit Gradient Boosting Regressor on } (X_{train}, y_{res, pilr})$ 
20:  for each target model in Models do
21:    Compute predictions across the complete spatial grid
22:    domain  $X$ 
23:    Calculate and store metrics: Train MSE, Test MSE,  $R^2$ ,
24:    Gen Gap, Physics Error
25:    Store raw grid evaluation coordinates in the global run
26:    log tracking system
27:  end for
28: end for

```

Algorithm 4 Bimodality Structural Analysis Loop

```

1: Compute and output the collective aggregate mean and variation
2:   standard deviations for all saved items
3: for each model configuration in Models do
4:   Initialize metric holders:  $\text{BIC}_{Single}$ ,  $\text{BIC}_{Double}$ ,  $\Delta \text{BIC}$ 
5:   for each execution sequence array recorded within the run logs
6:     do
7:        $\lambda_{extracted} \leftarrow \text{CALCULATE\_EFFECTIVE\_LAMBDA}(X, \text{prediction\_array})$ 
8:        $T_{1, distribution} \leftarrow 1.0 / \text{CLAMP}(\lambda_{extracted}, \min = 10^{-8})$ 
9:        $B_1 \leftarrow \text{Fit 1-Component Gaussian Mixture Model on } T_{1, distribution}$ 
10:       $B_2 \leftarrow \text{Fit 2-Component Gaussian Mixture Model on } T_{1, distribution}$ 
11:      Record derived scoring balances:  $\text{BIC}_{Single} \leftarrow B_1.\text{bic}()$ ,
12:       $\text{BIC}_{Double} \leftarrow B_2.\text{bic}()$ 
13:      Record structural separation step:  $\Delta \text{BIC} \leftarrow \text{BIC}_{Single} -$ 
14:       $\text{BIC}_{Double}$ 
15:    end for
16:  Compute and output configuration mean values paired with
17:  tracking sample error variation bounds
18: end for

```

XIII. Hardware Integration

This thesis is primarily computational and does not rely on explicit hardware implementation. However, it is motivated by an abstract correspondence with quantum-inspired systems, where a physical process can be viewed as a black-box mapping between inputs and outputs under unknown or implicit noise.

In this context, the simulation framework can be interpreted as a digital twin of such an underlying system. The objective is not to model physical device internals, but to learn an accurate functional approximation of its input-output behavior using data-driven models.

From a twin-technology perspective, the neural and hybrid models developed in this work act as surrogate systems that emulate the behavior of an unseen process. This enables efficient prediction and analysis without requiring direct interaction with physical hardware or detailed knowledge of its internal structure.

XIV. Computational Requirements

The experimental framework was executed entirely within a cloud-based environment using Google Colab, independent of the local host operating system. The cloud infrastructure consisted of an Intel Xeon CPU, an NVIDIA T4 GPU (15 GB VRAM), and 12.7 GB of system RAM running on a hosted Linux backbone.

XV. Relevant GitHub Links

References

- [1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [2] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
- [3] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*. Cambridge, U.K.: Cambridge Univ. Press, 2010.
- [4] P. Krantz, M. Kjaergaard, F. Yan, T. P. Orlando, S. Gustavsson, and W. D. Oliver, “A Quantum Engineer’s Guide to Superconducting Qubits,” *Applied Physics Reviews*, vol. 6, no. 2, p. 021318, 2019.
- [5] V. Sitzmann, J. N. P. Martel, A. W. Bergman, D. B. Lindell, and G. Wetzstein, “Implicit Neural Representations with Periodic Activation Functions,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 7462–7473, 2020.
- [6] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.
- [7] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear Partial Differential Equations,” *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.
- [8] K. P. Murphy, *Machine Learning: A Probabilistic Perspective*. Cambridge, MA, USA: MIT Press, 2012.
- [9] D. J. C. MacKay, *Information Theory, Inference, and Learning Algorithms*. Cambridge, U.K.: Cambridge Univ. Press, 2003.
- [10] S. M. Anton, C. Müller, J. S. Birenbaum, S. R. O’Kelley, A. P. Sears, and F. C. Wellstood, “Pure Dephasing in Flux Qubits Due to Flux Noise with Spectral Density Scaling as $1/f^\alpha$,” *Physical Review B*, vol. 85, no. 22, p. 224505, 2012.